Оглавление

Анализ файла 1
Статический анализ 1
Обфускация строк 1
Анализ функции selfID 1
Функция Ptrace 2
Функция main 3
Динамический анализ 3
Маппинг MITRE ATT&CK 4
Правила для auditd 4
IoC 5
Предотвращение 5

Анализ файла

Статический анализ

Обфускация строк

Для начала хочется отметить, что в коде была реализована обфускация строк засчёт XOR со 127. Эта реализация представлена в функции `XORdata`. Но также обращаю внимание, что по ходу всего кода не всегда использовалась эта функция. Также было просто XOR со 127 вне функции.

Анализ функции selfID

После анализа функция была переименована в `selfID`. В функции выводится ID процесса, а также вызывается функция `random_hex`, которая рандомизировано выводит hex последовательность с помощью /dev/random.

Это было сделано для того, чтобы засорить терминал, а также показать пользователю, что процесс легитимен, засчёт выполнения кода вывода hex и вывода ID процесса на экран.

```
void __noreturn selfID()
{
  __pid_t PID; // eax

  PID = getpid();
  printf("%u\n", PID);
  while ( 1 )
    random_hex();
}
```

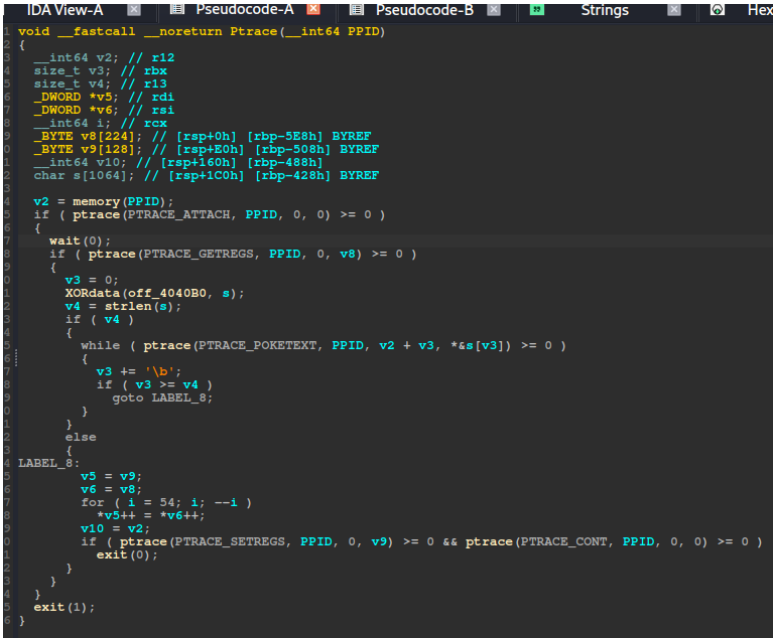
Функция selfID

Функция Ptrace

Логика этой функции заключается в том, чтобы внедрить код в память процесса.

Сначала вызывается функция `memory`, в которой происходит проверка атрибуции r-xp регионов памяти процесса-родителя (см. разбор функции `main`) с помощью чтения `/proc/%ld/maps`.

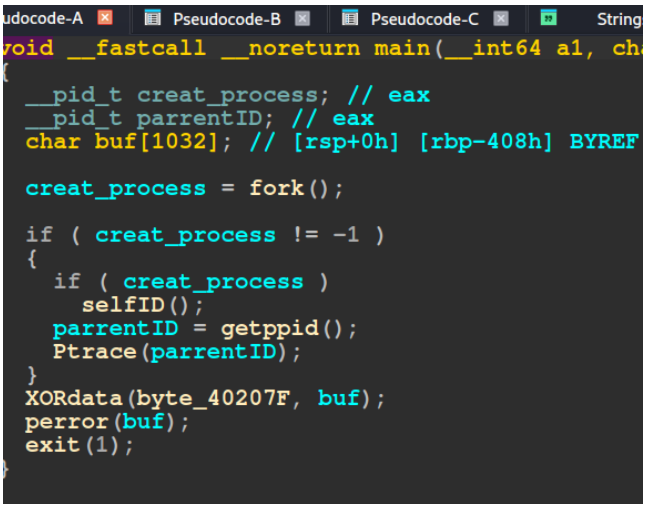
Код для внедрения лежит в `off_4040B0`. Внедрение кода происходит с помощью PTRACE_GETREGS (получения значений регистров процессора), PTRACE_POKETEXT (изменение памяти) и PTRACE_SETREGS (изменение регистров).



Функция Ptrace

Функция main

Теперь посмотрим, как эти функции работают сообща, и как работает `main`.



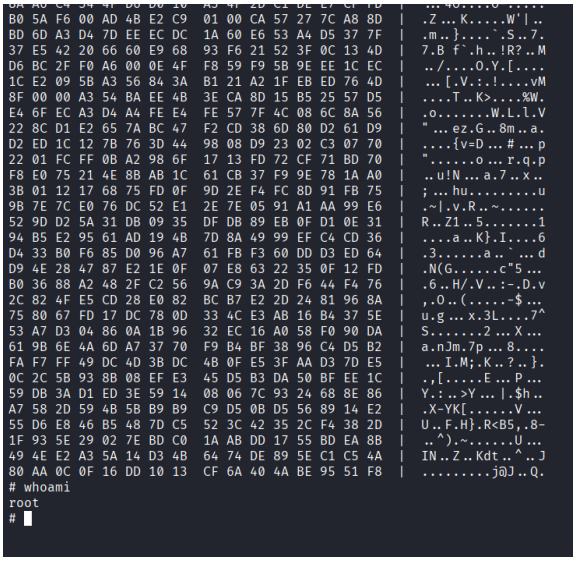
Функция main

В функции дочерний процесс узнаёт ID родительского процесса, чтобы потом через него внедрить обфусцированный шелл-код, через подключение `ptrace`. Для этого дочерний процесс узнаётся распределении памяти родительского процесса с правами доступа `r-xp` благодаря пути `/proc/<pid>/maps`.

Таким образом можно сделать вывод, что это загрузчик шелл-кода (Shellcode Loader / Injector) для систем Linux.

Динамический анализ

При запуске файла выводится случайные hex последовательности, а также запуск оболочки sh.



Запуск файла

Указатель off_4040B0 ведёт на данные в rodata по адресу 0x402063. При их расшифровке и дизассемблировании получаем код, который вызывает `/bin/sh` – командную оболочку. Однако оболочка не всегда может вызываться, поскольку если родительский процесс уже занят (например был запуск бинарника под gdb), то вывоз ptrace_attach от дочернего процесса к родительскому вернётся с ошибкой. Таким образом вызов ptrace служит и антиотладкой одновременно.

Маппинг MITRE ATT&CK

- Системный вызов ptrace: Process Injection: Ptrace System Calls
- Обфускация строк: Obfuscated Files or Information: Command Obfuscation

Правила для auditd

- Просмотр вызова ptrace:
  - PTRACE_POKETEXT
    - `auditctl -a always,exit -F arch=b64 -S ptrace -F a0=4 -k ptrace_memory_poketext`
    - `auditctl -a always,exit -F arch=b32 -S ptrace -F a0=4 -k ptrace_memory_poketext`
  - PTRACE_ATTACH
    - `auditctl -a always,exit -F arch=b64 -S ptrace -F a0=16 -k ptrace_process_attach`
    - `auditctl -a always,exit -F arch=b32 -S ptrace -F a0=16 -k ptrace_process_attach`
  - PTRACE_SETREGS
    - `auditctl -a always,exit -F arch=b64 -S ptrace -F a0=13 -k ptrace_process_setregs`
    - `auditctl -a always,exit -F arch=b32 -S ptrace -F a0=13 -k ptrace_process_setregs`
  - PTRACE_CONT
    - `auditctl -a always,exit -F arch=b64 -S ptrace -F a0=7 -k ptrace_cont`
    - `auditctl -a always,exit -F arch=b32 -S ptrace -F a0=7 -k ptrace_cont`
  - PTRACE_GETREGS
    - `auditctl -a always,exit -F arch=b64 -S ptrace -F a0=12 -k ptrace_getregs`
    - `auditctl -a always,exit -F arch=b32 -S ptrace -F a0=12 -k ptrace_getregs`
  - `auditctl -a always,exit -F arch=b64 -S process_vm_writev -k process_vm`
  - `auditctl -a always,exit -F arch=b32 -S process_vm_writev -k process_vm`

- Отслеживания памяти процессов:

    - `auditctl -a always,exit -F arch=b64 -S open,openat -F dir=/proc -k proc_access`
    - `auditctl -a always,exit -F arch=b32 -S open,openat -F dir=/proc -k proc_access`

IoC

Файловые

- /dev/random
- /proc/%ld/maps
- Хэш : d4e4b0839471f029d92a00d8cee8663b0586baf3be37dc5f6c4010237a177d6e
- XOR с 127

Поведенческие:

- getpid
- getppid
- ptrace
- XOR с 127

Предотвращение

Основная идея заключается в ограничении системного вызова ptrace. Возможность выполняться только определённым пользователям или процессам в системе. Также и для process_vm_readv/process_vm_writev

M1026 - Privileged Account Management: как способ ограничения ptrace. Это можно сделать через Yama (например, /proc/sys/kernel/yama/ptrace_scope).

Параметры:

- 0: ptrace разрешён любому процессу к любому
- 1: отлаживать процесс подключаться только к своим потомкам
- 2: только процессам с CAP_SYS_PTRACE (администратор)
- 3: отладка запрещена вообще для всех, включая root. После установки это значение sysctl изменить нельзя

Рекомендуемое значение 2 (если надо периодически использовать отладку администратору) или 3.

M1040 - Behavior Prevention on Endpoint: использовать системы защиты на конечных точек (например EDR).